s

Today

Welcome to your next lesson: Instance attributes vs class attributes.

I'm getting your personalized lesson ready — using what you've already learned, what's coming up, and the best way to help things stick.

Step by step, you’re moving forward.

Imagine an animal shelter 🐶🐱

The shelter has one shared rule: “This is a pet shelter.”
But each animal has its own details: name, age, health.

So we have:

Text
Shared for all pets:
- shelter type: "pet"

Different for each pet:
- name
- age
- health
In Python, “shared for all” will map to a class attribute, and “different for each one” will map to an instance attribute.

Quick check-in:

Does this real-world picture make sense so far, or is anything fuzzy about “shared vs individual”?

Yes, that makes sense

Great 👍

Now let’s connect that idea to code, step by step.

Here’s a simple Pet class:

Python

```
class Pet:
    kind = "pet"          # shared (class attribute)

    def __init__(self, name, age):
        self.name = name  # per-object (instance attribute)
        self.age = age    # per-object (instance attribute)
```

Visually:

Text

```
Pet class
└── kind = "pet"          ← shared by all pets

luna = Pet("Luna", 3)
└── name = "Luna"
└── age  = 3

max = Pet("Max", 5)
└── name = "Max"
└── age  = 5
```

kind → one value for the whole class
name, age → their own values for each instance
Make sense how kind is different from name/age here, in terms of shared vs per-pet?

Yes, that makes sense

Take your time with the question below and pick what you think is right 👇

In the Pet class below, which attribute is shared by all pets?

python

```
class Pet:
    kind = "pet"

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

A.
kind

B.
name

C.
age

D.
All of them are per-pet, none are shared

Nice work, Ron 🎯 You picked kind, which is exactly right.

kind = "pet" is defined once on the class, so it’s shared by all Pet objects.
self.name and self.age are stored inside each object, so they can be different for Luna, Max, etc.
Next step: I want to show you how reading these values works in practice (and how changing them can affect one pet vs all pets).

Are you at your computer and ready to use the code editor for a small experiment? 💻

Yes, I'm ready

Awesome 💪 Let’s do a tiny experiment with reading class vs instance attributes.

I’ll walk you through it step by step.

Open the Python code editor.
Paste this code in and run it (no changes yet):
Python

```
class Pet:
    kind = "pet"

    def __init__(self, name, age):
        self.name = name
```

```
        self.age = age


luna = Pet("Luna", 3)
max = Pet("Max", 5)

print("luna.kind:", luna.kind)
print("max.kind:", max.kind)

print("luna.name:", luna.name)
print("max.name:", max.name)
Run it first, then tell me:


Do luna.kind and max.kind print the same or different value?
Do luna.name and max.name print the same or different value?

You can just answer: “kind: same, name: different” (or
whatever you see).

kind prints the same name prints differnt

Perfect, you read that exactly right 🎯


luna.kind and max.kind are the same → they both use the
shared class attribute kind = "pet".
luna.name and max.name are different → they come from
instance attributes (self.name) stored separately in each
object.

So far you’ve seen:


Class attribute → one value, shared: kind
Instance attributes → per-object values: name, age
Next important idea: changing these values.


There are two different actions:


Pet.kind = "animal" → change the shared class attribute.
luna.kind = "dog" → give only Luna her own kind, which can
override the shared one just for her.

I’ll show you this in code next.
```

Before we do that: are you okay with moving on to “what happens when we change kind on the class vs on one pet”? 🐕‍🦺

Yes, let's try it

Great, let’s do it step by step 🧪

First, we’ll change the class attribute and see what happens.

Please open the code editor, replace what’s there with this, and run it:

```
Python
class Pet:
    kind = "pet"

    def __init__(self, name, age):
        self.name = name
        self.age = age


luna = Pet("Luna", 3)
max = Pet("Max", 5)

print("Before change:")
print("luna.kind:", luna.kind)
print("max.kind:", max.kind)

# Change the class attribute
Pet.kind = "animal"

print("After change:")
print("luna.kind:", luna.kind)
print("max.kind:", max.kind)
Run it, then tell me (in words):
```

After the change, do luna.kind and max.kind still match each other?
And what value do they show now?
yes they still match now as animals

Exactly right ✅

By doing Pet.kind = "animal" we changed the one shared value on the class:

Before: both printed "pet"
After: both print "animal"

So:

kind is shared class state → one change on Pet.kind affects all pets that don’t have their own kind.
name, age stay per-object → nothing about them changed.
Next, I want to show you the opposite: changing kind only for Luna using luna.kind = "dog" and seeing how that affects Luna vs Max.

Ready to try that version? 🐕🐕

Yes, let's try it

Nice 🔍 Now we’ll see how only Luna can get her own kind.

Step-by-step:

Open the editor again.
Replace the code with this and run it:
Python

```
class Pet:
    kind = "animal"

    def __init__(self, name, age):
        self.name = name
        self.age = age


luna = Pet("Luna", 3)
max = Pet("Max", 5)

print("Before change:")
print("luna.kind:", luna.kind)
print("max.kind:", max.kind)
```

```
# Give ONLY Luna her own 'kind'
luna.kind = "dog"

print("After change:")
print("luna.kind:", luna.kind)
print("max.kind:", max.kind)
Run it, then tell me:


After the change, what does luna.kind print?
After the change, what does max.kind print?
luna,kind prints dog max.kind prints animal

Perfect, Ron — that’s the key behavior 🔑


After luna.kind = "dog":


luna.kind → "dog" (Luna now has her own kind, stored on the
instance)
max.kind → "animal" (Max still uses the shared class
attribute from Pet.kind)

So:


Pet.kind = shared class attribute
luna.kind = "dog" = instance attribute that shadows the class
one, but only for Luna

Visually:

Text
Pet class
└── kind = "animal"      (shared default)

luna
├── name = "Luna"
├── age  = 3
└── kind = "dog"         (Luna's own value: shadows class)

max
├── name = "Max"
└── age  = 5             (no own 'kind' → uses class 'animal')
I want to check your understanding in your own words:

```

When would you choose to make something a class attribute instead of an instance attribute?

Just a short answer like: “I’d use a class attribute when …”

when every copy of it made will share the same attribute

That’s a great way to say it 🙌

Yes: use a class attribute when every object made from that class should share the same value.

Use an instance attribute when each object might have its own value.

Quick “attribute detective” game 🔍

For a Pet shelter, say if each one should be shared (class) or per-pet (instance):

Shelter name (e.g., "Happy Tails Shelter")
Pet name (e.g., "Luna")
Health level (e.g., 80 out of 100)
Species (all are pets; assume this shelter is only for pets)

You can answer like:

“Shelter name: class, Pet name: instance, …”

shelter name: per instance, pet name: per instance, health level: per instance, species, shared class

Nice reasoning, and you nailed an important one with species as shared 👏

Let’s tune the others a bit:

Shelter name → usually shared (class): all pets in that program belong to the same shelter.
Pet name → per instance: every pet needs its own name.
Health level → per instance: each pet’s health can be

different.
Species / kind (like "pet" here) → shared (class) if all pets are the same category.

So the rule you gave earlier still stands:

If every object should have the same value → class attribute.

If it can differ per object → instance attribute.

Want to do a quick non-pet example next (like Car or Student) to practice choosing class vs instance attributes? 🚗🎓

different name foe each shelter though sane variable name but unique to each as i see it

Ah, I see what you're thinking, and you're right to question that 💡

You're imagining a world with many shelters, each with its own name — so each pet might belong to a different shelter. In that design:

Shelter name per pet → would indeed be an instance attribute (because it can differ per pet).

In our small example, we were modeling one shelter program where every pet is from the same shelter, so I treated shelter_name as shared class data.

So the deeper rule stays the same:

Don't think "class or instance?"

Think: "In this program's world, does this value stay the same for all objects, or can it vary?"

Same variable name, but where you put it (class vs instance) depends on the world you're modeling.

Want to try a quick practice where we design a Car class and you decide which values are class vs instance?

Yes, let's try the car example

Nice, let's keep it simple with Car 🚗

Think of a basic Car class. Here are some possible pieces of data:

Text
- brand (e.g., "Toyota")
- model (e.g., "Corolla")
- number_of_wheels (usually 4)
- color (e.g., "red")

Step by step, using your rule:

If every car of this class in our program shares it → class attribute
If it can be different for each car → instance attribute
How would you classify these four for a normal program with several cars?

You can answer like:

brand: ?
model: ?
number_of_wheels: ?
color: ?

theres more then one brand of car so instance theres more then one model, instance, were assuming no odd euro 3 wheel cars so class for number of wheels, and theres many unique colors so instance arttribute

That's excellent reasoning, Ron 🔥 You're really applying the rule well.

Given the world you described (many brands/models/colors, but still 4 wheels):

brand → instance (each car can have a different brand)
model → instance (varies with each specific car)
number_of_wheels → class (shared fact: all our cars in this program have 4 wheels)
color → instance (each car can be painted differently)

So for this particular design, we might write:

Python
```
class Car:
    number_of_wheels = 4   # class attribute (shared)

    def __init__(self, brand, model, color):
        self.brand = brand   # instance
        self.model = model   # instance
        self.color = color   # instance
```
You're clearly getting the "ownership decision" idea: look at the world you're modeling, then decide shared vs per-object.

To close this lesson, I want to give you one short multiple-choice question about pets again, to lock it in. Ready for a quick check?

Ready for the quick check

What do you think? Pick the best answer in the question box 👇

A pet shelter has this Python class:

python
```
class Pet:
    shelter_policy = "All pets must be vaccinated"

    def __init__(self, name, health):
        self.name = name
        self.health = health
```
Which piece of data should be a shared class attribute, not an instance attribute?

A.
The shelter policy text: "All pets must be vaccinated"

B.
Each pet's health value

C.
Each pet's name value

D.
None of them should be shared; all should be per-pet

You nailed that one 💯

Shelter policy → same rule for every pet → perfect as a class attribute (shelter_policy).
name and health → clearly per-pet details → correct to keep as instance attributes.

You've shown you can:

Tell shared class state from per-object instance state
Decide where data belongs based on the world you're modeling (shelter, cars, etc.)

That's exactly what this lesson was about 🎯

Quick recap in your own words (optional, but powerful):

If a friend asked, "When should I use a class attribute vs an instance attribute in Python?" how would you answer in 1–2 sentences?

If you're done for now, you can also just head to the next lesson whenever you're ready.